

Variáveis Globais: Grande Perigo Silencioso!

Mário Leite

Variáveis globais parecem práticas. Elas ficam “à mão”, podem ser acessadas por qualquer função e, no começo, dão a sensação de simplificar o programa. Mas essa facilidade cobra um preço alto: qualquer parte do código pode alterar um valor importante sem que o programador perceba.

Imagine um sistema financeiro simples, onde existe uma variável global chamada **saldo**, representando o dinheiro real de um cliente. A função principal registra um saque legítimo:

```
saldo inicial: R$ 5000
saque:         R$ 1000
saldo atual:    R$ 4000
```

Pronto. O desastre aconteceu.

O saldo real do cliente, que era **R\$ 4000**, foi substituído por **R\$ 30**. Depois, quando o programa cobra a tarifa, ele subtrai **R\$ 20** desse valor corrompido. Resultado final:

```
saldo exibido:  R$ 10
saldo correto:  R$ 3980
dinheiro “sumido”: R$ 3970
```

E o pior: o erro não dá mensagem, não trava o programa, não aparece como falha de sintaxe. Ele simplesmente produz um resultado errado. Esse é o tipo mais perigoso de erro: **o erro silencioso**.

A lição é clara: **variáveis globais aumentam o risco de efeitos colaterais**. Uma função que deveria apenas consultar ou imprimir informações pode acabar modificando dados essenciais do programa inteiro.

A boa prática é simples:

- prefira variáveis locais;
- passe valores por parâmetro;
- devolva resultados com retorno;
- evite que funções alterem estados globais sem necessidade;
- use nomes claros para variáveis temporárias;
- trate dados críticos, como saldo, estoque e notas, com cuidado especial.

Variáveis globais não são proibidas; em alguns casos, até são úteis: constantes, *arrays*, registros, configurações gerais, estados controlados do sistema. Mas devem ser usadas com extrema cautela e muita sabedoria.

Em programação, o problema nem sempre é o erro que aparece na tela. Muitas vezes, o maior perigo é o erro que passa despercebido.

Moral da história: uma variável global mal usada pode transformar uma simples função de impressão em uma máquina de destruir dados. Observe a saída na **Figura 1**.

```

"""
PerigoGlobalFinanceiro.py
=====

Demonstra o PERIGO CATASTRÓFICO de variáveis globais num cenário
financeiro: uma função que SÓ deveria IMPRIMIR o comprovante acaba
CORROMPENDO o saldo global do cliente, provocando um rombo silencioso.

RASTRO DO DESASTRE (saída esperada):
saldo inicial          = 5000.0
após o saque (legítimo) = 4000.0
[ a função corrompe saldo para 30.0 por dentro ]
após cobrar a tarifa    = 10.0  <-- saldo que aparece na tela
saldo que DEVERIA ser   = 3980.0
DINHEIRO QUE SUMIU      = 3970.0  <-- o rombo
=====
"""

# constante GLOBAL (Em Python, a convenção para constantes é CAIXA ALTA)
TARIFA = 20.00

# variável GLOBAL: visível em TODO o programa (eis o perigo)
saldo = 0.0

def principal():
    # Em Python, é obrigatório usar 'global' para modificar a variável fora do escopo
    global saldo

    # Variáveis locais
    saque = 0.0
    esperado = 0.0
    rombo = 0.0

    print()
    print("=====")
    print("  DEMONSTRAÇÃO: O PERIGO DAS VARIÁVEIS GLOBAIS")
    print("  Cenário: conta bancária com saldo GLOBAL")
    print("=====")
    print()

    saldo = 5000.00
    print(f"[1] SALDO INICIAL DO CLIENTE: R$ {saldo}")
    print()

    saque = 1000.00
    print(f"[2] Cliente solicita um saque de R$ {saque}")
    saldo = saldo - saque      # débito legítimo do saque
    print(f"[3] Saque debitado. Saldo agora: R$ {saldo}")
    print()

    print(f"[4] Emitindo o comprovante (função que SÓ deveria imprimir)...")
    emitir_comprovante(saque)  # <<< O PROGRAMADOR ACHA QUE É INOFENSIVO
    print()

    print(f"[5] Cobrando a tarifa de serviço do saldo...")
    saldo = saldo - TARIFA     # ELE ACHA QUE O SALDO AINDA É 4000...
    print()

    esperado = 5000.00 - saque - TARIFA  # o que o cliente deveria ter
    rombo = esperado - saldo             # o que sumiu sem explicação

```

```
print("!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!")
print(f"    SALDO FINAL QUE APARECE NA TELA: R$ {saldo}")
print(f"    SALDO QUE O CLIENTE DEVERIA TER: R$ {esperado}")
print(f"    DINHEIRO QUE SUMIU DA CONTA : R$ {rombo}")
print("!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!")
print()

print("CONCLUSAO:")
print(" A função emitir comprovante() usou a variável GLOBAL 'saldo'")
print(" como se fosse um rascunho local. Ao fazer isso, APAGOU o")
print(" saldo real do cliente (R$ 4000) e o trocou por R$ 30.")
print(" Como 'saldo' é global, a principal() continuou operando sobre")
print(" o valor corrompido, e o cliente perdeu R$ 3970 sem ninguém")
print(" entender o porquê. É ASSIM QUE UMA VARIÁVEL GLOBAL CAUSA")
print(" UM DESASTRE.")
print()
print(" MORAL: se 'saldo' fosse LOCAL (ou passado por parâmetro),")
print(" a função NUNCA conseguiria corrompê-lo por acidente.")

# -----
# Pelo NOME, esta função SÓ deveria IMPRIMIR o comprovante.
# Mas ela enxerga a GLOBAL 'saldo' e a usa como rascunho: é aí que mora o perigo.
# -----

def emitir_comprovante(saque):
    # O programador declarou 'global saldo' aqui para poder "montar o total"
    # no rodapé do comprovante, sem perceber que estava sobrescrevendo o estado real.
    global saldo

    iof = saque * 0.01          # IOF do saque = 10.00

    print("    --- COMPROVANTE ---")
    print(f"    Valor do saque : R$ {saque}")
    print(f"    IOF                : R$ {iof}")
    print(f"    Tarifa             : R$ {TARIFA}")
    print(f"    (saldo visto aqui, antes do bug: R$ {saldo})") #ainda 4000.00

    # >>> BUG DE ESCOPO GLOBAL <<<
    # O programador precisou de um "total" para o rodapé do comprovante,
    # mas NÃO declarou uma variável local (ex: total_encargos): reutilizou 'saldo'
    # achando que era um rascunho inofensivo. Como 'saldo' é GLOBAL, o saldo REAL
    # do cliente (R$ 4000) é APAGADO e substituído por 30.00 neste instante.
    saldo = iof + TARIFA        # saldo global vira 30.00 !!!

    print(f"    Total de encargos no rodapé: R$ {saldo}")
    print("    --- FIM COMPROVANTE ---")

# Programa Principal-----
# Ponto de entrada do programa em Python
if __name__ == "__main__":
    principal() #fim do programa "PerigoGlobalFinanceiro"
```

```
IDLE Shell 3.13.15
File Edit Shell Debug Options Window Help
Python 3.13.15 (tags/v3.13.15:4061bc4, Aug 5 2026, 13:05:39) [MSC v.1944 64 bit
(AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
= RESTART: H:/Drive D/Cantinho da Programação/Códigos/Python/PerigoGlobalFinanceiro.py

=====
DEMONSTRAÇÃO: O PERIGO DAS VARIÁVEIS GLOBAIS
Cenário: conta bancária com saldo GLOBAL
=====

[1] SALDO INICIAL DO CLIENTE: R$ 5000.0

[2] Cliente solicita um saque de R$ 1000.0
[3] Saque debitado. Saldo agora: R$ 4000.0

[4] Emitindo o comprovante (função que SÓ deveria imprimir)...
--- COMPROVANTE ---
Valor do saque : R$ 1000.0
IOF           : R$ 10.0
Tarifa        : R$ 20.0
(saldo visto aqui, antes do bug: R$ 4000.0)
Total de encargos no rodapé: R$ 30.0
--- FIM COMPROVANTE ---

[5] Cobrando a tarifa de serviço do saldo...

!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
SALDO FINAL QUE APARECE NA TELA: R$ 10.0
SALDO QUE O CLIENTE DEVERIA TER: R$ 3980.0
DINHEIRO QUE SUMIU DA CONTA : R$ 3970.0
!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!

CONCLUSAO:
A função emitir_comprovante() usou a variável GLOBAL 'saldo'
como se fosse um rascunho local. Ao fazer isso, APAGOU o
saldo real do cliente (R$ 4000) e o trocou por R$ 30.
Como 'saldo' é global, a principal() continuou operando sobre
o valor corrompido, e o cliente perdeu R$ 3970 sem ninguém
entender o porquê. É ASSIM QUE UMA VARIÁVEL GLOBAL CAUSA
UM DESASTRE.

MORAL: se 'saldo' fosse LOCAL (ou passado por parâmetro),
a função NUNCA conseguiria corrompê-lo por acidente.
>>> |
```

Ln: 44 Col: 0

Figura 1 - Saída do programa "PerigoGlobalFinanceiro"